

A Simple Blueprint for Building Software with AI Agents

Stealing the best of Spec Kit, Superpowers and GStack into one workflow a data scientist can actually follow — from idea to shipped.

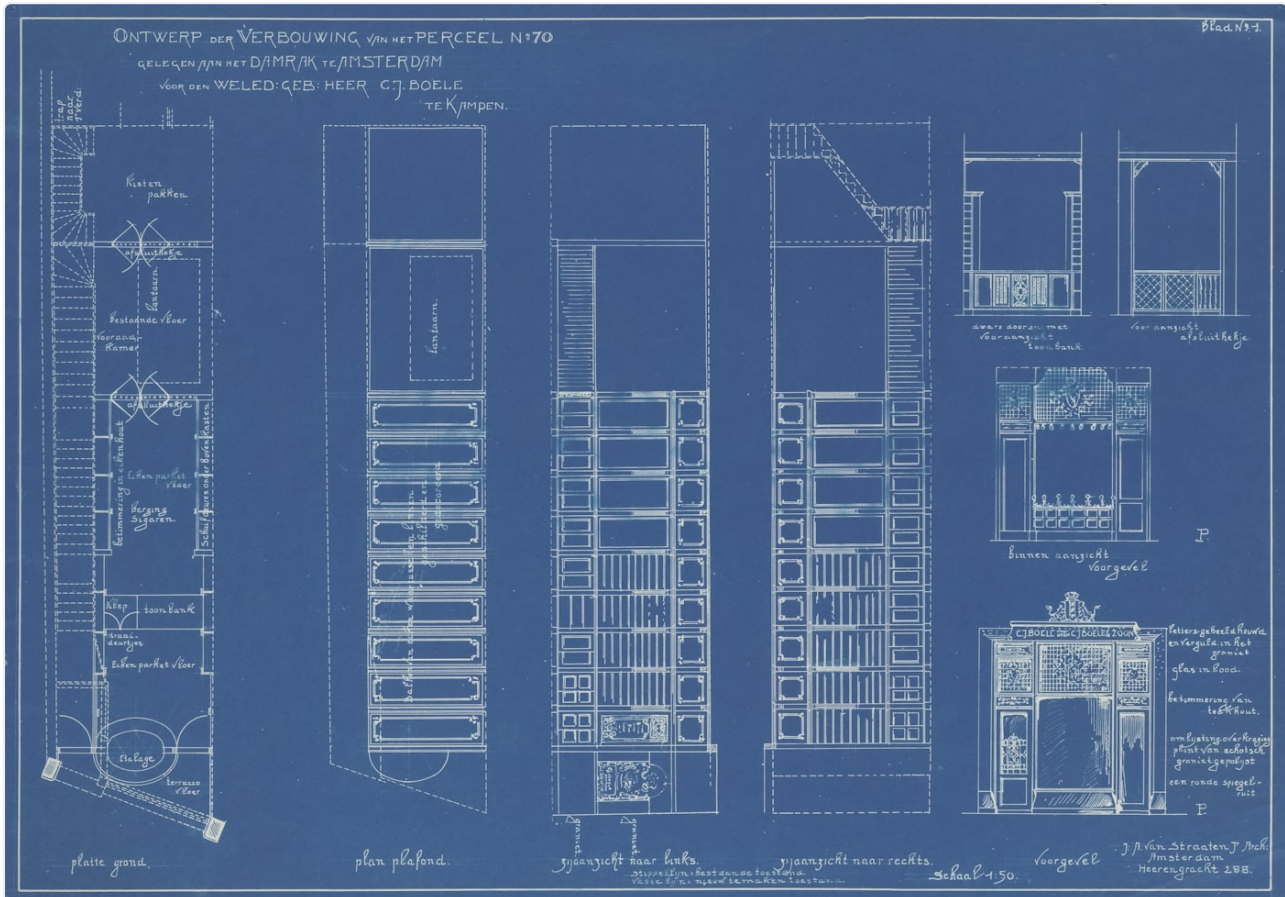


Photo by Amsterdam City Archives on Unsplash

I'll be honest about where I'm coming from. I've spent years in data science, and my "development process" for most of that time was a notebook, a lot of cells run out of order, and a general faith that I'd remember what I did last Tuesday. It works — right up until the thing you built has to survive being used by someone who isn't you. Data scientists mostly learn to model. We rarely learn the software development lifecycle that surrounds the model: the spec, the plan, the review, the tests, the boring paperwork that stops a Friday deploy from becoming a Saturday incident.

Then AI coding agents showed up, and with them a wave of frameworks that promise to bolt a real engineering process onto the agent. Three kept coming up in my setup — **Spec Kit**, **Superpowers**, and **GStack**. Each is genuinely good. Each also has its own vocabulary and, between them, something like sixty slash-commands. For someone who just wants to take an idea and ship it responsibly, that is a lot of choice to stare at on a Monday morning.

So I did what I do with any new tool: tried them, got confused, and then tried to distill the confusion into one workflow I could actually remember. This article is that blueprint — a simplified, opinionated path from ideation to shipped, borrowing the best stage from each framework and saying plainly which I'd reach for and why.

There's a picture that captures the whole reason this matters. In a demo, building with an agent is two boxes: you prompt it, you get working code, everyone claps. Shipping the same thing for real is the same build with the wishful thinking removed — a whole stack of stages the agent will happily skip if you let it. The frameworks exist to put those stages back.

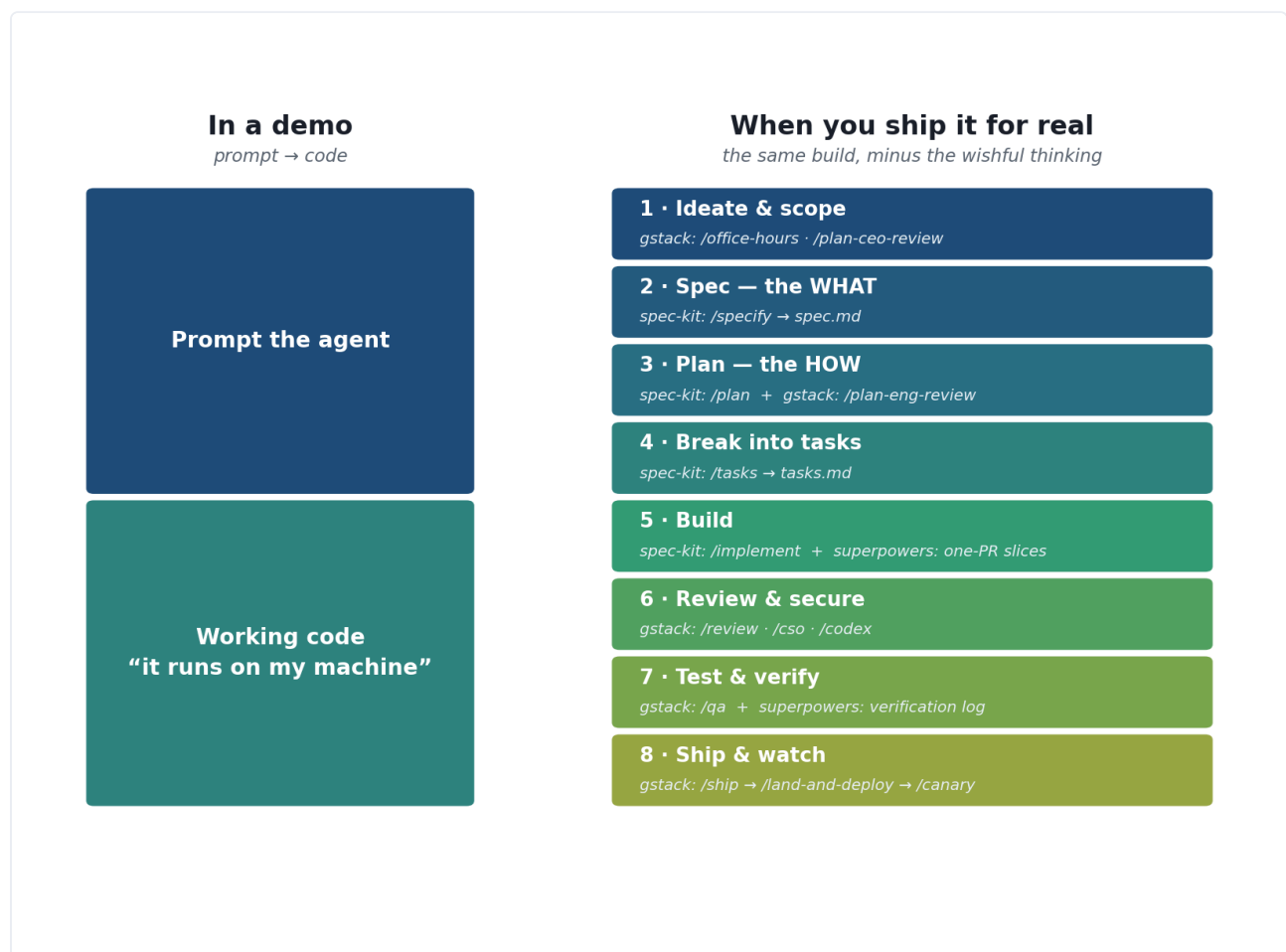


Image by Author

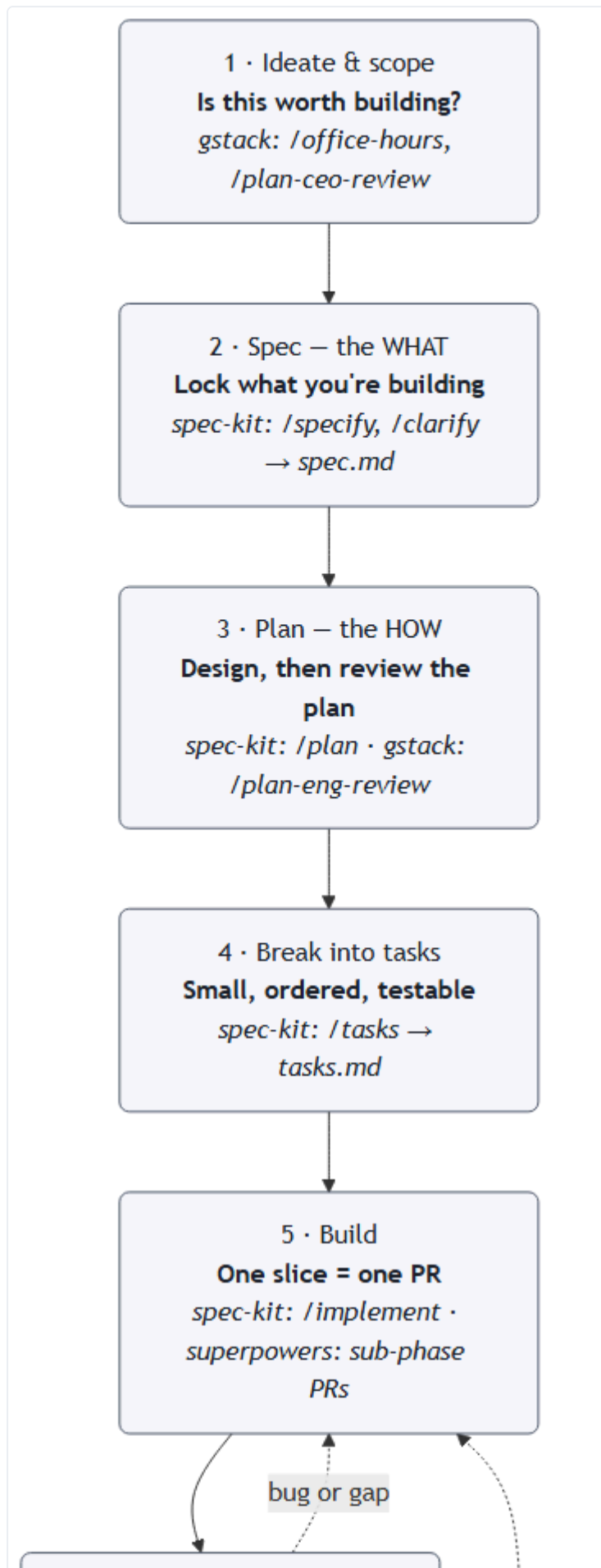
The remarks below are my personal take basis usage and how each tool actually executes — not a benchmark, and no branding or promotion here. Your mileage will vary with your stack and how much ceremony your project deserves.

A one-minute primer on the three frameworks

Before the blueprint, the three players in plain language:

- **Spec Kit** — GitHub’s open-source toolkit for *spec-driven development*. Its whole philosophy is: decide **what** you’re building before **how**, and write it down. It gives you a pipeline of Markdown artifacts — `spec.md` → `plan.md` → `tasks.md` — that each feed the next, so the agent codes against a document instead of a vibe. It’s the most widely adopted of the three and the closest thing to an industry standard here.
- **Superpowers** — Jesse Vincent’s (obra) agentic skills framework. Its obsession is *engineering-grade* output: real test-driven development, small independently-shippable slices, and — my favourite idea — an **append-only verification log** that records what was tested, on which commit, with the evidence. It’s less a set of commands and more a discipline.
- **GStack** — a full-cycle *review-and-ship* toolkit. This is the one with a command for every stage: brainstorming (`/office-hours`), scope review (`/plan-ceo-review`), engineering review (`/plan-eng-review`), live browser QA (`/qa`), security audit (`/cso`), and the whole ship-deploy-monitor tail (`/ship` , `/land-and-deploy` , `/canary`).

Here’s the thing that took me a while to see: **they don’t compete, they slot together**. Spec Kit is the planning engine. GStack is the review-and-ship engine. Superpowers is the discipline that keeps the build honest. The blueprint below is mostly about picking the right one for each stage — and being clear about the overlaps.



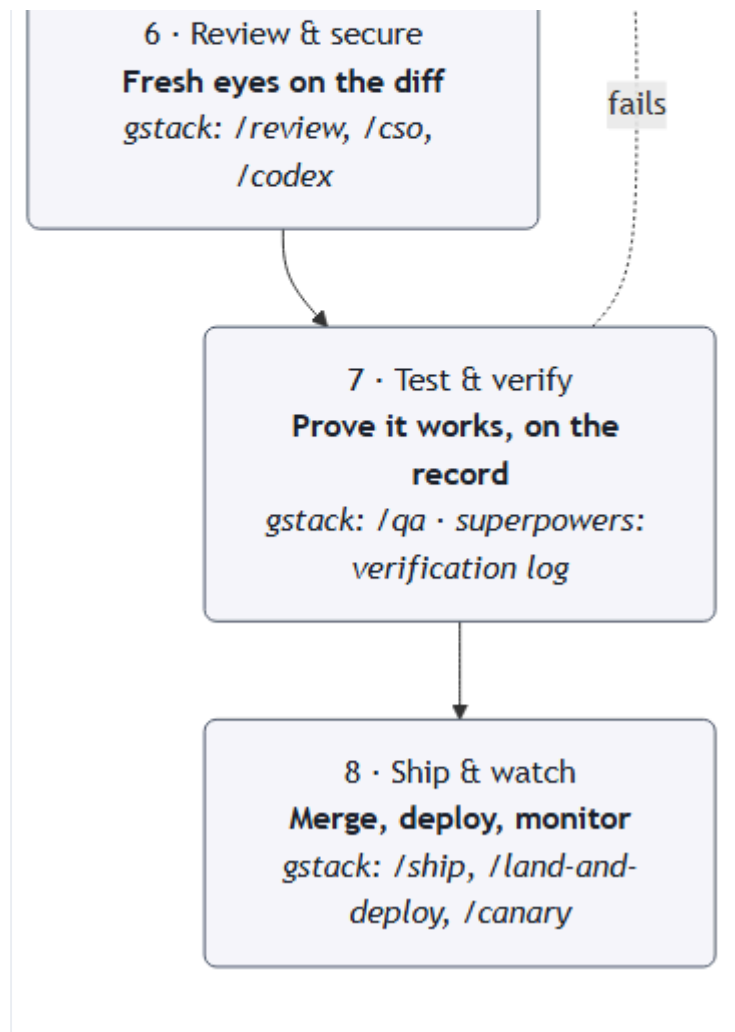


Image by Author

Stage 1 — Ideate and scope: is this even worth building?

The most expensive code is the code you shouldn't have written. Before any spec, the useful question is whether the idea survives contact with a hard “why”.

GStack owns this stage cleanly with `/office-hours` — it challenges the premise, forces you to name the problem, and offers A/B/C alternatives instead of rubber-stamping your first idea. Follow it with `/plan-ceo-review` when scope is fuzzy and you need someone to argue for cutting it down.

Neither Spec Kit nor Superpowers really does this part — they assume you've already decided to build. So there's no contest here: **GStack, because it's the only one that treats “should we?” as a first-class step.** As a data scientist this was the stage I most wanted to skip and most needed to keep.

Artifact: a short design doc capturing the problem, the alternatives considered, and what you deliberately chose

not

to do.

Stage 2 — Spec: lock down the WHAT

This is Spec Kit's home turf and the single habit that changed my output the most. You run `/specify`, describe the feature in plain English, and it writes a `spec.md` — user stories, success criteria, edge cases — with *no implementation detail*. Then `/clarify` scans it for ambiguity and makes you quantify the hand-wavy words ("fast" becomes "under 200ms"). It caps the interrogation at a handful of questions so it doesn't turn into a survey.

Superpowers also does a "locked decisions" brainstorm, but it builds on the same spec-driven idea, and Spec Kit is more popular and more turnkey for this. **So: Spec Kit, for the maturity of the artifact and the fact that a spec written this way is one an agent can actually execute against.**

The trade-off is real, though — this stage adds friction. For a one-file weekend script it's overkill. For anything another human will touch, the spec pays for itself the first time someone asks "wait, what was this supposed to do?".

Artifact: `spec.md` — the contract for what you're building, and nothing about how.

Stage 3 — Plan: the HOW, then a second pair of eyes

Now `/plan` (Spec Kit) turns the spec into a technical plan — architecture, data model, the interfaces — as `plan.md`. It'll do a little research phase first for the decisions it isn't sure about.

Here's where the frameworks genuinely overlap, and where I make my one firm recommendation: **let Spec Kit produce the plan, but let GStack review it.** Run `/plan-eng-review` (and `/plan-design-review` if there's a UI) on the freshly written `plan.md`. Spec Kit's own `/analyze` checks that your documents are *consistent with each other*; GStack's eng-review asks the harder question of whether the plan is any *good* — edge cases, test coverage, performance. Two different jobs. Catching an architecture problem here, on a Markdown file, is orders of magnitude cheaper than catching it in code.

Artifact: `plan.md` , plus a review pass that leaves review notes and a test plan behind.

Stage 4 — Break it into tasks

`/tasks` (Spec Kit) turns the plan into `tasks.md` : a dependency-ordered checklist of small, independently-testable steps, with parallel-safe ones marked. There's no real competition for this — it's a mechanical, valuable step that Spec Kit does well. The point is less *which tool* and more the principle: a big feature becomes a list of small, checkable things, and both you and the agent always know what “done” means for each one.

Artifact: `tasks.md` — the build broken into checkboxes.

Stage 5 — Build: one slice at a time

Now the agent codes. Spec Kit's `/implement` walks the task list phase by phase, ticking items off as it goes. That's the executor.

But *how* you build is where I steal from Superpowers. Its discipline — **one meaningful slice equals one pull request**, forward-only changes, test-driven where it counts — is the difference between a diff a human can review and a 4,000-line blob nobody will. For anything non-trivial, don't let the agent build the whole feature in one shot. Slice it the Superpowers way and land each slice on its own.

So this stage is a genuine mix: **Spec Kit to execute the tasks, Superpowers' PR-slicing discipline to keep the build reviewable**. The trade-off is speed versus control — small PRs are slower to ship and much faster to trust.

Artifact: working code, landed as small reviewable slices rather than one heroic commit.

Stage 6 — Review and secure: fresh eyes on the diff

An agent reviewing its own code has the same blind spots as a person marking their own exam. This is why the review stage wants a

different

voice, and it's where GStack is strongest. Three commands, three jobs:

- `/review` — a diff review of the unmerged change, looking for the bugs a plan-review couldn't see because the code didn't exist yet.
- `/codex` — an independent second opinion from a different model. Cheap insurance; it catches things one model consistently misses.
- `/cso` — a dedicated security audit (OWASP-style). As a data scientist this is the muscle I'd never built, and having security be its own *named stage* rather than a vague good intention is the whole value.

Spec Kit's `/analyze` also lives near here, but remember it audits your *documents*, not your code — useful as a cheap pre-check, not a substitute. **For actual code and security review: GStack, and treat security as a step you don't get to skip.**

Artifact: review findings fixed, and a security pass on the record.

Stage 7 — Test and verify: prove it, and write down the proof

Two things that sound the same and aren't. *Testing* is exercising the thing; *verification* is the durable record that you did. I'd use both frameworks here, deliberately.

For testing, GStack's `/qa` drives a real browser through every route, finds bugs, and fixes them iteratively — far closer to how software is actually tested than a data scientist's usual "it ran without erroring, ship it". For the record, I'd borrow Superpowers' **append-only verification log**: a table stamped with the commit SHA, the command run, expected-versus-actual, and a link to the evidence. The rule that makes it powerful is that you never edit past rows — a wrong-then-corrected entry *is* the audit trail.

So: GStack to test, Superpowers to verify. The log feels like bureaucracy until the first time someone asks "was this actually tested before it went out?" and you can answer with a commit hash instead of a shrug.

Artifact: bugs fixed, plus a verification log entry proving what passed and when.

Stage 8 — Ship and watch

The unglamorous tail that data science training skips entirely. GStack owns it end to end:

`/ship` commits, pushes and opens the PR; `/land-and-deploy` merges and deploys; `/canary` watches production afterward for errors and regressions. Neither Spec Kit nor Superpowers

claims this ground — their job ends at “verified and ready”. **GStack, uncontested, because shipping and watching is a first-class stage, not an afterthought.**

Artifact: a merged PR, a live deploy, and a monitor watching it.

Food for thought

A few things I’m still chewing on, offered as open questions rather than settled answers:

Ceremony vs. speed: the full eight-stage path is right for a feature real users depend on. It is absurd for a throwaway script. The skill isn’t running every stage — it’s knowing which stages a given piece of work has actually earned. I don’t have a clean rule for that yet.

Tool sprawl: three frameworks and sixty commands is itself a cost. The blueprint here is my attempt to hide that behind eight decisions, but the honest version is that a beginner still has to install and reason about all three. A single framework that did all eight stages this well would beat any mix-and-match.

The agent’s blind spots: almost every stage above exists because an AI agent, left alone, will confidently skip it. Spec, review, security, verification — these are guardrails against an over-eager builder. As the models get better, which of these stops being necessary? I genuinely don’t know.

Learnings and where I’d go next

- **The process is the point, not the tool.** Coming from data science, the real unlock wasn’t any one command — it was internalising that *ideate* → *spec* → *plan* → *build* → *review* → *secure* → *test* → *ship* is a sequence with a reason for each step. The frameworks just make each step cheap.
- **Spec Kit plans, GStack ships, Superpowers keeps you honest.** If you remember one thing, remember which engine owns which job.
- **Security and verification were my two biggest gaps,** and they’re exactly the two stages a modeller’s training never covers. Making them named, non-skippable steps did more for me than any amount of better prompting.
- **Next for me:** wiring this into a single reusable checklist so I stop deciding the workflow from scratch each time — and testing where the “skip it for small work” line actually sits. If I get that right, I’ll write it up.

If you're a data or ML person who's always felt slightly fraudulent about the "software" half of the job — this blueprint is the scaffolding I wish I'd had. Steal it, cut the stages your work hasn't earned, and make it yours.

References

1. Spec Kit — GitHub's Spec-Driven Development toolkit. <https://github.com/github/spec-kit>
2. Spec-Driven Development with AI — The GitHub Blog. <https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/>
3. Superpowers — Jesse Vincent (obra), an agentic skills framework & software development methodology. <https://github.com/obra/superpowers>
4. "Superpowers: How I'm using coding agents" — Jesse Vincent, Massively Parallel Procrastination. <https://blog.fsck.com/2025/10/09/superpowers/>
5. GStack — the full-cycle review-and-ship skill collection referenced here is from my own Claude Code setup. *[Add public link if/when published.]*